

Northstar — Team Operations Platform

Product & Engineering Documentation

Document status: Internal working documentation

Audience: Product, Engineering, Design, QA, Operations

Product: Northstar

Release context: Initial production release and subsequent operational expansion

1. Product Overview

Northstar is intended to become the primary workspace through which growing organizations coordinate recurring operational work.

The original motivation for the product came from teams that had gradually accumulated a mixture of spreadsheets, email conversations, shared documents, chat messages, and small internal applications. None of these tools was particularly problematic on its own. The difficulty appeared when information needed to move between them. A request might begin in a conversation, become an item in a spreadsheet, acquire supporting material in a shared folder, and eventually be discussed again in another conversation several weeks later.

This made it difficult to answer seemingly simple questions.

Who currently owns a piece of work? Has it been completed? Why was a particular decision made? Which customer or internal department originally requested it? What changed since the previous review? Who was responsible when an issue occurred?

Northstar approaches the problem by giving each organization a persistent workspace in which operational work can be represented as projects and work items. The system is not intended to replace every communication or document-management tool used by an organization. Instead, it should provide a reliable center around which those other activities can be organized.

The initial product is aimed primarily at organizations with between ten and several hundred active users. It is expected that a single organization may contain several teams with different responsibilities. Not every employee should necessarily have the same ability to modify information.

The product should feel relatively lightweight for ordinary members while providing enough administrative controls for organizations that need stronger governance.

There is an important distinction between information that represents the current state of work and information that explains how that state was reached. For example, changing a

work item's owner is a current-state operation, whereas the fact that the item was previously owned by another person may be useful for later investigation. The product therefore needs to preserve certain historical information rather than treating all data as disposable current state.

Northstar is expected to be accessed primarily through a browser. Mobile-specific behavior is not part of the initial product definition, although the underlying API should not make mobile clients impossible later.

2. Terminology

The product uses several terms that have specific meanings.

An **organization** is the top-level boundary within which users, projects, settings, and operational data exist.

A **member** is a user who has access to an organization. A person may belong to more than one organization using the same account.

A **workspace** is the environment presented to members after they enter an organization. In most product conversations, organization and workspace are used almost interchangeably, although organization is preferred when discussing ownership and administration.

A **project** represents a body of related work. Projects normally correspond to initiatives, customer engagements, operational programs, or other bounded areas of responsibility.

A **work item** is an actionable unit belonging to a project. Users often call these tasks, but the API and database terminology uses work item.

A **comment** is a discussion attached to a work item.

An **attachment** is a file associated with a work item or comment.

A **role** describes what a member is permitted to do. Roles are evaluated within an organization; the same person can therefore have different responsibilities in different organizations.

An **activity** represents a significant change or event that the system believes may be useful to understand later.

The product uses “delete” carefully. Removing something from an active interface does not necessarily mean that the underlying information should cease to exist.

3. Accounts and Identity

Northstar accounts are based around an email address, although the email address should not be considered an organization's identity for authorization purposes.

A person joining two organizations should continue using the same account rather than creating a second account for each organization. This becomes especially relevant for consultants and contractors who may work with several customers.

The registration experience is intentionally simple. A new user provides an email address and password and is subsequently asked to verify ownership of that address. Until verification is completed, the account may exist but should not be treated as fully active.

The team has discussed allowing users to sign in through external identity providers. That is not required for the initial release. The account model should nevertheless avoid assumptions that an account must always authenticate using a password.

Email addresses are treated case-insensitively for account lookup. The address displayed to the user may preserve the capitalization originally supplied, but two differently capitalized representations of the same address should not result in separate accounts.

A password reset begins from the sign-in experience. A user who no longer remembers their password should be able to request assistance without an authenticated session. The reset mechanism should not reveal whether a particular email address belongs to an existing account. This matters because the login system should not become a simple directory of registered users.

Password-reset links are temporary. Once a reset has been successfully completed, an old reset link should not remain usable.

Sessions should be invalidated when appropriate security-sensitive actions occur. In particular, changing a password should not leave every previously issued session permanently trusted.

The system should avoid storing raw passwords or password-reset secrets.

Users should be able to change their own profile information. Display name is used throughout the application and should not be forced to equal the email address.

A user may deactivate their account. Account deactivation has implications for organizations to which the user belongs, and the organization should not unexpectedly lose historical references merely because the individual is no longer active.

4. Organizations

The first user creating an organization becomes its initial owner.

An organization has a name and may optionally have a short identifier used in URLs and integrations. The identifier should be unique enough to avoid ambiguous routing, but it should not be assumed that an organization's display name is unique.

Organizations are isolated from one another. A member of Organization A should not gain access to Organization B merely because both organizations contain the same email address or because an object identifier happens to be known.

This isolation extends beyond the web interface. API authorization must enforce the same boundary.

An organization owner can invite additional people. Invitations are sent to email addresses rather than requiring the administrator to know whether the recipient already has a Northstar account.

If the recipient already has an account, accepting an invitation should add the existing account to the organization. It should not create a duplicate identity.

If the recipient does not yet have an account, the invitation can become part of the registration process.

Invitations should not remain valid indefinitely. The organization administration area should provide enough information for an administrator to understand which invitations are still awaiting action.

An invitation sent to an incorrect address may need to be revoked. Revocation should prevent the invitation from subsequently being accepted.

It is possible for the same person to receive invitations from multiple organizations. Accepting one must not consume or invalidate unrelated invitations.

Organizations may have their own timezone setting. This affects how dates and scheduled events are presented. Stored timestamps should remain unambiguous even when different members view the same information from different locations.

The organization name may be changed later. References in historical activity should remain understandable after such changes.

5. Membership and Roles

Not everyone in an organization should be able to administer it.

The initial product has three broad levels of access: owner, administrator, and member.

The owner is responsible for organization-level control. There is normally one owner at a time, although ownership can be transferred.

Administrators can manage many operational aspects of the organization but should not be able to perform actions reserved exclusively for the owner.

Ordinary members can participate in projects and work but should not be presented with organization-wide controls merely because they know the relevant URLs.

Authorization is therefore not purely a matter of hiding buttons. Server-side checks remain necessary.

Removing a member from an organization should immediately prevent that member from accessing organization data through normal authenticated requests.

Removal should not rewrite old activity. If an individual changed the status of a work item three months ago, later removing them from the organization should not cause that activity to lose its historical association.

Likewise, work items that were assigned to a removed member should not silently disappear. Existing records may continue to show the person as the historical assignee. Whether the item should be reassigned is a separate organizational decision.

An administrator may change a member's role, subject to restrictions around ownership.

A member cannot elevate their own permissions by modifying a request sent to the server.

The organization member list should make it possible to distinguish active members from pending invitations.

The product team does not currently want a large arbitrary permission matrix. The initial role model should remain understandable enough that an administrator can explain it to another employee without consulting technical documentation.

6. Projects

Projects provide the primary grouping mechanism for operational work.

A project has a name, description, creator, owner, status, timestamps, and membership information.

Not every organization member necessarily needs to participate in every project. Project access should therefore be considered separately from organization membership.

When a project is created, the creator normally becomes its initial project owner.

Project owners can add organization members to the project. A person outside the organization should not be added directly to a project.

The product team expects projects to remain useful after their active work has ended. For this reason, completing or archiving a project is preferable to permanently deleting it in most ordinary workflows.

Archived projects should no longer appear alongside active projects in the primary project-selection experience. However, users with sufficient access should still be able to locate and inspect archived projects.

The distinction between completed and archived work is intentional. A completed project represents work that reached its intended conclusion. Archiving is primarily an organizational operation that removes a project from active working views.

The exact visual treatment is a design concern, but the underlying state needs to distinguish these concepts.

Project names do not need to be globally unique. Two different projects in the same organization may have similar names if the organization chooses to allow that.

Deleting a project is considered a destructive operation and should therefore not be part of casual workflows.

Where project deletion is supported, the system must consider dependent work items, comments, attachments, and activity records rather than leaving orphaned data behind.

Project descriptions may contain ordinary formatting. They should not be interpreted as executable content.

A project can have a due date, but the absence of a due date is valid. Dates should not be invented merely because a project was created.

7. Project Membership

Project membership determines which organization members participate in a particular project.

A project owner can add and remove members. Organization administrators may need equivalent capabilities for operational reasons.

Removing someone from a project is different from removing them from the organization.

A removed project member should lose access to project-specific information that is restricted to project participants. However, information that the organization explicitly makes broadly visible may remain available.

Existing work assigned to the removed member should not automatically be deleted.

There is also no general expectation that removing a member should automatically reassign every piece of work. In many organizations, reassignment requires a human decision because the correct replacement depends on context.

When someone leaves a project, their previous comments and recorded activities remain part of the project's history.

A project owner should not be removable through an ordinary member-removal action without another owner being designated.

The interface should avoid allowing an organization to reach a state where an active project has no responsible owner unless the product explicitly supports such a state.

8. Work Items

Work items are the units through which teams coordinate actual work.

A work item belongs to one project.

At minimum, a work item has a title and status. Additional information includes description, priority, creator, assignee, timestamps, and optional due date.

The title is required because many parts of the application use it as the compact representation of the work item.

Descriptions are optional. A work item consisting only of a short title is valid.

The status model should remain intentionally small in the first release. The product team has found that teams often invent complicated workflows when given too many states, but those states rarely remain consistent.

The initial workflow consists of an item that has not yet been completed, an item currently being worked on, and an item that has been completed.

Users frequently want to move items backward as circumstances change. Completing an item therefore should not make it permanently immutable.

An item marked complete can subsequently return to an active state.

Priority is represented using a small ordered range rather than arbitrary numeric values. The interface should communicate the meaning of each level rather than requiring users to remember numeric codes.

A work item may have no assignee. Unassigned work is legitimate, especially during planning.

Only organization members who can access the relevant project should normally be available as assignee candidates.

If an assignee is subsequently removed from the project, existing assignment information should not be destroyed merely to make the current member list cleaner.

Due dates are dates rather than precise moments for ordinary work items. A task due on a particular day remains due on that calendar date regardless of whether the viewer is in a different timezone.

The system should distinguish between an absent due date and a due date that happens to be today.

Work items can be reordered within project views. The ordering is intended to reflect user preference rather than imply business priority.

The application should not assume that a higher-priority item must appear first unless the selected view explicitly sorts by priority.

9. Work Item Editing

Most fields can be changed after creation.

The creator of an item does not automatically retain permanent ownership of every later change. Project permissions govern whether another user can modify it.

When a title is changed, historical activity should make it possible to understand that a change occurred. It is not necessary to preserve every previous character sequence indefinitely.

Status changes are particularly important because teams use them to understand movement through a project.

Changing an assignee is also worth recording because ownership is often relevant when reviewing incidents.

A user should be able to remove an optional due date after it has been set.

Descriptions may be edited without creating a separate version of the entire project.

The API should reject malformed identifiers and invalid state values rather than silently converting them.

Requests involving an object belonging to another organization should behave as unauthorized or unavailable rather than exposing whether the object exists.

10. Comments and Discussion

Work items include a discussion area.

Comments are intended for contextual conversation rather than as a replacement for a full messaging system.

A comment has an author, body, creation time, and potentially an edited state.

Users can edit their own comments. Organization administrators may require the ability to moderate comments in exceptional circumstances.

Deleting a comment should not make the activity history misleading. The interface can indicate that content was removed without pretending that the comment never existed.

The product team has deliberately avoided building threaded discussion in the first release. Comments therefore belong directly to the work item.

Comments may include basic formatting, but the stored representation must be treated as untrusted input.

The application should escape or otherwise safely render user-supplied content. A user entering markup into a comment should not gain the ability to execute code in another user's browser.

The team expects comments to become useful evidence during later reviews. Consequently, author and timestamps are important even when the comment text is eventually edited.

11. Attachments

Users sometimes need to provide supporting documents, screenshots, spreadsheets, or other files alongside a work item.

Attachments may be uploaded directly to a work item and may also be associated with an individual comment.

The exact file storage mechanism is considered an implementation concern. The application should not assume that uploaded content belongs on the same server as the application itself.

There should be reasonable limits around uploaded content. Extremely large files are outside the intended use case and should be rejected rather than allowed to consume unlimited storage.

File names are supplied by users and should therefore not be trusted as filesystem paths.

Two files may have the same display name without representing the same attachment.

Deleting an attachment should remove it from normal access. Historical activity can indicate that an attachment was removed without retaining a publicly accessible copy of the file.

An attachment belonging to a work item must follow the access rules of that work item. Knowing a storage identifier should not be sufficient to retrieve a private file.

Downloads should be authorized independently from the page that displays the work item.

The product team has not committed to inline preview for every file type. Unsupported formats may simply be offered as downloadable content.

12. Notifications

Northstar should notify people when something relevant happens to their work.

Notifications are primarily intended to prevent users from needing to repeatedly inspect every project for changes.

Being assigned a work item is normally notification-worthy.

Being mentioned in a comment is also notification-worthy.

A comment on an item that a person is actively following may generate a notification depending on the notification settings available to that user.

Not every database change should become a notification. For example, internal bookkeeping changes that have no user-facing significance should remain invisible.

Notifications should identify the event sufficiently for a user to understand why it appeared.

A notification can be marked as read without deleting it.

The notification list should distinguish unread items from previously viewed ones.

Users may have preferences controlling certain categories of notification. The system should not interpret disabling an email notification as disabling the underlying event or removing an in-app notification unless the product explicitly defines those as the same preference.

Email delivery may fail independently of the application. A failed email should not cause the underlying assignment or mention to be rolled back.

The system should avoid producing duplicate notifications when the same operation is retried.

13. Email Delivery

Transactional email is used for account verification, password recovery, invitations, and selected notifications.

Emails contain links back to Northstar. Links involving authentication or invitations should contain temporary credentials or tokens rather than exposing passwords.

Email templates may be updated independently of the main application logic.

The application should not assume that email delivery is instantaneous.

An invitation may remain pending while the email provider is experiencing delays.

Where practical, email sending should occur asynchronously so that a user does not have to wait for an external mail service before receiving confirmation that an in-app action succeeded.

Email addresses should not be logged unnecessarily in places where operational logs are broadly accessible.

The system should distinguish between an action that successfully created an invitation and an attempt to deliver the associated email.

14. Search

As organizations accumulate projects and work items, browsing becomes increasingly inefficient.

Northstar therefore includes search across accessible operational content.

Search should respect the same organization and project access boundaries as ordinary page requests. Search is not an exception to authorization.

A user searching for a phrase should not discover a project they otherwise cannot open.

The initial search experience focuses on project names, project descriptions, work item titles, work item descriptions, and comments.

Search results should provide enough surrounding context for users to recognize why a result was returned.

Search is not expected to understand natural-language questions in the first release.

The product team prefers useful partial matching over requiring users to know exact titles.

Search indexing may not be instantaneous. A newly created item can therefore take a short period before appearing in results.

Deleting or restricting an object should eventually remove it from search results available to users who no longer have access.

Search ranking is intentionally not treated as a contractual ordering. The system should prioritize useful results, but clients should not depend on one exact ranking algorithm.

15. Dashboard

The dashboard is intended to answer the question, “What needs my attention?”

It should summarize active projects and work assigned to the current user.

Completed and archived work should not dominate the default dashboard.

A user with no assigned work should still receive a useful empty state rather than a technical error.

The dashboard may include recently updated items. “Recently updated” refers to meaningful user-facing changes, not every database timestamp modification.

The organization timezone should be considered when presenting date-oriented information.

Dashboard counts should be derived from current accessible data rather than being permanently trusted as independent truth. If an item is removed or becomes inaccessible, the dashboard should eventually reflect that.

The dashboard is a convenience view, not a separate data model for work.

16. Activity History

Organizations occasionally need to reconstruct how a project reached its current state.

Northstar therefore records significant activities.

Examples include creation of a project, changes to project ownership, addition or removal of project members, creation of work items, assignment changes, status changes, and certain administrative actions.

Activity should include the actor where an identifiable actor exists.

Some system-generated activities may have no human actor.

Activities should contain timestamps that can be interpreted consistently.

The activity feed is not intended to record every internal database operation. Updating a search index, refreshing a cache, or retrying an email should not normally appear as user activity.

Historical information should remain intelligible if the display name of the actor changes later.

If a user is deactivated or removed, historical activity should continue to refer to the relevant identity without granting that identity new access.

Activity records are append-oriented. The system should avoid allowing ordinary users to rewrite history simply by modifying current objects.

Administrators may have broader access to activity information than ordinary project participants.

The team expects audit-style information to become more important as larger customers adopt the platform.

17. Dates, Times, and Timezones

Time handling has caused problems in previous internal tools, so Northstar should avoid ambiguous representations.

Timestamps representing events should have an unambiguous timezone interpretation.

The organization has a configured timezone used for organization-oriented calendar displays.

Users may view the application from another timezone, but this should not cause an event to be stored as a different moment.

Work-item due dates have different semantics. A due date is associated with a calendar day rather than an exact instant.

For example, if a work item is due on October 14, a user traveling across timezones should not see the due date unexpectedly become October 13 simply because their local clock is different.

Relative labels such as “today” and “tomorrow” should be generated using the relevant user or organization context rather than being permanently stored as text.

Daylight-saving transitions should not cause recurring or date-based logic to assume that every calendar day has exactly the same number of hours.

18. Organization Settings

Organization administrators can configure basic workspace behavior.

The organization name and timezone are among the primary settings.

Some settings may eventually govern notification behavior or security policies.

Settings should be scoped to the organization rather than accidentally becoming global user preferences.

Changing an organization's timezone affects how future displays interpret dates and times. It should not rewrite the original timestamps of existing activities.

Settings changes may themselves be recorded in administrative activity.

An organization should retain sensible defaults when a setting has never been explicitly configured.

The system should distinguish an absent optional configuration from a deliberately selected default where that distinction matters operationally.

19. Administration

The administration area provides organization-level controls.

It includes membership management, pending invitations, organization settings, and selected security information.

Administrators should be able to inspect current members and their roles.

The owner has additional controls around ownership and organization lifecycle.

Administrative pages must not be accessible simply because a user knows their URL.

An administrator who loses administrative privileges during a session should not continue to perform administrative operations indefinitely because an old browser page remains open.

Sensitive operations may require additional confirmation.

The product team wants destructive operations to feel deliberate. This includes removing members, deleting projects where deletion is available, and transferring ownership.

The interface should make it difficult to mistake “remove from project” for “remove from organization.”

20. Ownership Transfer

Organizations are expected to outlive individual employees.

Ownership therefore needs to be transferable.

A current owner may designate another active organization member as owner.

The recipient must already belong to the organization.

Ownership transfer changes organizational responsibility; it does not create a second organization.

The previous owner may remain an administrator or member depending on the role selected during the operation.

The operation should be recorded because later administrators may need to establish when ownership changed.

A removed or deactivated account should not leave the organization permanently ownerless if the product's supported lifecycle allows administrators to recover ownership.

Recovery procedures may require elevated operational access and are not intended as an ordinary member workflow.

21. Data Visibility

Visibility follows a layered model.

At the broadest level, organization membership determines whether a user belongs to the tenant.

Projects can further restrict access.

Work items inherit the visibility of their project unless a future feature introduces more granular controls.

Comments and attachments follow the visibility of their parent object.

This inheritance is important because it prevents developers from accidentally implementing separate authorization rules for every child object.

An API endpoint that returns a comment should verify access to the parent work item.

Similarly, downloading an attachment should not bypass the project membership check merely because the attachment has its own identifier.

Public sharing links are not part of the initial release.

The product should therefore not have a concept of an unauthenticated user accessing a private project simply because someone forwards them a URL.

22. API Considerations

The web application communicates with a backend API.

The API should use resource identifiers that are safe to expose to clients. Sequential numeric identifiers are not prohibited, but authorization must never depend on the difficulty of guessing an identifier.

Clients should receive consistent representations for common resources.

Validation errors should be distinguishable from authorization failures and server failures.

A malformed request should not produce an internal stack trace or implementation details in the normal API response.

The API may evolve over time. Client code should not need to depend on undocumented database fields.

For operations that modify resources, clients should receive enough information to update their local view without immediately reloading every page.

Where an operation can reasonably be retried, the backend should avoid accidentally creating duplicate logical records.

This matters particularly for invitation creation, notification generation, file metadata creation, and other actions that may be triggered again after a network timeout.

Pagination is expected for collections that can grow substantially, including organization members, projects, notifications, search results, and activity feeds.

The server should not assume that a client will always request the first page.

Ordering of paginated results needs to be stable enough that records are not randomly skipped or repeated as the user moves between pages.

23. Concurrency

Multiple people may work on the same project simultaneously.

The application should tolerate two users editing different aspects of the same work item without corrupting unrelated information.

A user viewing stale information should not be able to unknowingly overwrite newer changes in situations where the distinction matters.

The exact concurrency strategy is an engineering decision.

For simple fields, last-write behavior may be acceptable in some cases. For operations such as membership changes or ownership transfer, stronger guarantees are likely necessary.

The system should be especially careful with actions that must happen only once.

For example, an invitation should not accidentally result in two memberships because the acceptance request was submitted twice.

Similarly, two concurrent attempts to transfer ownership should not result in an invalid state with conflicting owners.

Retries from clients and infrastructure are expected and should be treated as normal rather than exceptional behavior.

24. Reliability

The application should be designed on the assumption that individual dependencies occasionally fail.

Email providers can become unavailable.

Object storage can temporarily reject requests.

Database connections can fail.

Search indexing can lag.

None of these should automatically imply that unrelated core operations are rolled back.

For example, creating a work item should not fail merely because the search indexing service is temporarily unavailable, provided the work item can be safely persisted and indexed later.

Likewise, a notification delivery problem should not undo a successful assignment.

Background processing should therefore be capable of retrying certain operations.

Retries should not create duplicate user-visible side effects.

Operations that cannot be retried safely should either use an idempotency mechanism or otherwise establish whether the original operation succeeded before performing it again.

25. Background Processing

Some work is deliberately separated from the request that initiated it.

Sending email, updating search indexes, generating certain notifications, and processing uploaded files are candidates for asynchronous processing.

The user interface should distinguish between a completed primary operation and background processing where that distinction matters.

A newly uploaded file might initially have a processing state before becoming available for normal download.

Background jobs should have enough information to identify their target organization and resource without relying on whatever browser session happened to trigger them.

Jobs may be retried.

A job that repeatedly fails should not retry forever without visibility to operators.

Operational tooling should allow failed background work to be investigated.

26. Files and Security

Uploaded files are untrusted.

A filename such as `../../something` must never become a direct filesystem location.

File content should not automatically be interpreted as HTML or executable code merely because a browser can display that format.

The application should use appropriate content-disposition behavior for downloads where inline rendering could introduce security concerns.

Authentication and authorization must be applied before generating or serving private file access.

Temporary download mechanisms may be used if the storage architecture requires them.

A temporary file-access mechanism should have a limited lifetime.

File upload limits should be enforced before allowing unbounded data transfer.

The system should consider the possibility that two users upload files with identical names and that one file may be replaced while another remains valid.